

Assignment 2 (10% of total marks)

Due date: Sunday, 10 May 2026 by 9:00 pm Singapore time

Scope:

The tasks of this assignment cover **index, stored PL/pgSQL procedure, function, and trigger**. The assignment covers the topics discussed in lecture 5, 6, 7, and 8.

Assessment criteria:

Marks will be awarded for:

- Correct,
- Comprehensive, and
- Appropriate

application of the materials covered in this subject.

Assignment Specification:

Preliminary actions

Step 1

If you have not done it yet, download the TPCHR sample database and load the sample TPCHR using user account tpchr. This will make the user account tpchr the owner of the TPCHR database.

Step 2

In this step we shall use the relational tables included in a sample TPCHR benchmark database owned by the user tpchr. The conceptual schema of the sample database is included in the file tpchr.pdf.

Task 1 (7.0 marks)

Stored PL/pgSQL Procedure

Main learning objectives

- COUNT()
- grouping by supplier
- average comparison
- handling integer-based aggregates

Task Specification:

Implement a stored PL/pgSQL procedure **SUPPLIERPARTCOUNT** that lists suppliers in a specified region and counts how many different parts each supplier supplies.

The procedure should calculate the average number of parts supplied per supplier in each nation and indicate whether each supplier supplies above or below the nation average.

Input

```
CALL SUPPLIERPARTCOUNT('ASIA');
```

Output should include

Nation name: VIETNAM

Supplier name	Phone	Number of parts supplied	Comment
Supplier#000000884	31-445-893-2210	34	The supplier supplies above the nation average of 28 parts.
Supplier#000001022	31-288-154-9911	21	The supplier supplies below the nation average of 28 parts.

Tables involved

- supplier
- partsupp
- nation
- region

Submission Instructions:

Submit two files:

1. A2SolutionT1.sql – Contains the full PL/pgSQL procedure implementation.
2. A2SolutionT1.pdf – Contains screenshots of terminal output, including:
 - SQL statements executed
 - Procedure output
 - Ensure there are no errors in the displayed results

SQL Script Requirements:

- At the beginning of your script, include the following:
 - **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
 - **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- Your **report must satisfy all requirements**; any submission failing to meet them may receive **no marks**.
- Format monetary values consistently using TO_CHAR or equivalent functions (if any).
- Handle **invalid or non-existent regions** gracefully with an informative message.
- Include **all SQL statements and execution output** in your report.

Sample solution includes but is not limited to the following:

```
CREATE OR REPLACE PROCEDURE SUPPLIERPARTCOUNT(p_nation_name VARCHAR)
LANGUAGE plpgsql
AS $$
DECLARE
    v_nation_id INTEGER;
    v_avg_parts NUMERIC(10,2);
    v_supplier_record RECORD;
BEGIN
    -- Get the nation key for the specified nation name
    SELECT N_NATIONKEY INTO v_nation_id
    FROM NATION
    WHERE N_NAME = TRIM(p_nation_name);

    -- Check if nation exists
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Nation "%" not found', p_nation_name;
```

```
END IF;

-- Calculate the average number of parts supplied per supplier in the nation
SELECT COALESCE(AVG(part_count), 0) INTO v_avg_parts
FROM (
    SELECT S.S_SUPPKEY, COUNT(DISTINCT PS.PS_PARTKEY) AS part_count
    FROM SUPPLIER S
    JOIN PARTSUPP PS ON S.S_SUPPKEY = PS.PS_SUPPKEY
    WHERE S.S_NATIONKEY = v_nation_id
    GROUP BY S.S_SUPPKEY
) AS supplier_parts;

-- Output header
RAISE NOTICE 'Nation name: %', TRIM(p_nation_name);
RAISE NOTICE '';
RAISE NOTICE 'Supplier name      Phone      Number of parts supplied  Comment';
RAISE NOTICE '-----';

-- Get each supplier and their part count with comparison comment
FOR v_supplier_record IN
    SELECT
        TRIM(S.S_NAME) AS supplier_name,
        TRIM(S.S_PHONE) AS phone,
        COUNT(DISTINCT PS.PS_PARTKEY) AS part_count
    FROM SUPPLIER S
    JOIN PARTSUPP PS ON S.S_SUPPKEY = PS.PS_SUPPKEY
    WHERE S.S_NATIONKEY = v_nation_id
    GROUP BY S.S_SUPPKEY, S.S_NAME, S.S_PHONE
    ORDER BY S.S_NAME
LOOP
    RAISE NOTICE '% % %  %',
        v_supplier_record.supplier_name,
        v_supplier_record.phone,
        v_supplier_record.part_count,
        CASE
            WHEN v_supplier_record.part_count > v_avg_parts
            THEN format('The supplier supplies above the nation average of %s parts.', ROUND(v_avg_parts))
            ELSE format('The supplier supplies below the nation average of %s parts.', ROUND(v_avg_parts))
        END;
END LOOP;

-- If no suppliers found
IF NOT FOUND THEN
    RAISE NOTICE 'No suppliers found for nation: %', TRIM(p_nation_name);
END IF;
END;
$$;
```

tpchr=> -- Call the procedure

NOTICE: -----

NOTICE: Supplier#000000023 19-559-422-5776 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000045 19-189-635-8862 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000050 19-561-560-7437 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000051 19-475-537-1368 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000089 19-259-876-1014 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000153 19-346-843-5260 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000162 19-905-888-6020 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000168 19-985-776-2090 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000189 19-482-239-6669 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000194 19-741-822-3512 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000219 19-708-346-1944 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000225 19-787-123-9863 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000250 19-347-403-1571 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000278 19-563-887-6379 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000293 19-143-962-9484 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000313 19-648-945-5128 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000332 19-777-809-6033 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000364 19-653-706-8164 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000394 19-517-731-4139 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000403 19-513-268-3908 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000439 19-415-856-7881 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000455 19-822-816-5632 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000469 19-527-692-4725 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000481 19-951-947-8520 80 The supplier supplies below the nation average of 80 parts.

NOTICE: Supplier#000000505 19-480-691-1853 80 The supplier supplies below the nation average of 80 parts.

Task 2 (3.0 marks)

Stored PL/pgSQL Function

Task Specification:

Objective:

Implement a stored PL/pgSQL function that finds the **cheapest** and **most expensive** parts within a specified part type.

The function should accept a part type as its input parameter and return a formatted string containing both results.

Scenario and Requirements

You are to create a stored function GET_PARTPRICE_RANGE that performs the following:

1. **Accepts** a part type (e.g., 'STANDARD POLISHED STEEL') as its input parameter.
2. **Identifies** the part with the *lowest* retail price and the part with the *highest* retail price within that type.
3. **Returns** a formatted string displaying:
 - The part key
 - The part name
 - The retail price for both the cheapest and the dearest parts

Expected Output Example

Cheapest part: 143, Part#000000143, \$57.25.

Most expensive part: 1225, Part#000001225, \$986.75.

Assumptions

- The database schema contains the TPC-H-style table:
- part(p_partkey, p_name, p_type, p_retailprice, ...)
- The p_retailprice field represents the retail price of the part.
- You may assume that at least one part exists for a given type.

Example Usage:

```
-- Example calls to the function
SELECT get_partprice_range('ECONOMY POLISHED STEEL');
SELECT get_partprice_range('STANDARD POLISHED STEEL');
SELECT get_partprice_range('PROMO BRUSHED COPPER');
```


Example Output

Cheapest part: 143, Part#000000143, \$57.25.

Most expensive part: 1225, Part#000001225, \$986.75.

Execution Instructions:

- **PL/pgSQL Script:**
 - Write and save the stored function implementation in a PL/pgSQL script named A2SolutionT2.sql.
- **Execution:**
 - Run the function using a SELECT statement to perform the computations.
- **Execution Output:**
 - Capture all terminal output, including success and error messages, in a file named: A2SolutionT2.txt
 - Save the captured output in a **PDF file**: A2SolutionT2.pdf

Submission Instructions:

You are required to submit two files:

1. **SQL script file** (A2SolutionT2.sql) containing:
 - CREATE FUNCTION statement
 - The test SELECT statements
2. **Execution report** showing:
 - Successful creation of the function
 - Results of all test queries
 - No syntax or runtime errors

Report Guidelines:

- Your report must include:
 - **Screenshots of the SQL output:** Copy and paste the screen output from your terminal into a document, then save it as a PDF file.
 - **SQL statements processed:** The report must list **all SQL statements** that were executed.
 - Ensure there are **no errors** in the output shown in your report.

SQL Script Requirements:

- At the beginning of your script, include the following:

- **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
- **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- Your **report must satisfy all requirements**; any submission failing to meet them may receive **no marks**.
- Format monetary values consistently using TO_CHAR or equivalent functions (if any).
- Handle **invalid or non-existent regions** gracefully with an informative message.
- Include **all SQL statements and execution output** in your report.

Sample solution includes but is not limited to the following:

```
CREATE OR REPLACE FUNCTION get_partprice_range(p_type TEXT)
RETURNS TEXT AS $$
```

```
DECLARE
```

```
    v_min_partkey  INTEGER;
    v_min_partname TEXT;
    v_min_price    NUMERIC(15,2);
```

```
    v_max_partkey  INTEGER;
    v_max_partname TEXT;
    v_max_price    NUMERIC(15,2);
```

```
    v_result      TEXT;
```

```
BEGIN
```

```
-- Find the cheapest part within the given type
SELECT p_partkey, p_name, p_retailprice
INTO v_min_partkey, v_min_partname, v_min_price
FROM part
WHERE p_type = p_type
ORDER BY p_retailprice ASC
LIMIT 1;
```

```
-- Find the most expensive part within the given type
SELECT p_partkey, p_name, p_retailprice
INTO v_max_partkey, v_max_partname, v_max_price
FROM part
WHERE p_type = p_type
ORDER BY p_retailprice DESC
LIMIT 1;
```

```
-- Construct the formatted output
```

```
v_result := format(
    'Cheapest part: %s, %s, $%s.%sMost expensive part: %s, %s, $%s.',
    v_min_partkey, v_min_partname, to_char(v_min_price, 'FM999,999,999.00'),
    chr(10), -- newline
    v_max_partkey, v_max_partname, to_char(v_max_price, 'FM999,999,999.00')
);

RETURN v_result;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN format('No parts found for type "%s".', p_type);
    WHEN OTHERS THEN
        RETURN format('An error occurred: %s', SQLERRM);
END;
$$ LANGUAGE plpgsql;
```

Task 3 (7.0 marks)

Stored PL/pgSQL Trigger

Task Specification:

Scenario:

In the LINEITEM table, l_tax must be between 0 and 1.

You are required to create a **row-level trigger** that:

Validates the l_tax value prior to an insert or update operation. If the value is outside the range [0, 1], it must be normalized: values less than 0 are set to 0, and values greater than 1 are set to 1.

Learning focus:

- BEFORE INSERT OR UPDATE
- Numeric validation
- Trigger-based correction of invalid data
- Modifying NEW values

Trigger Details:

- The trigger should fire **before a new lineitem record is inserted** or **an existing record is updated**.
- It should inspect the l_tax value:

- If NEW.l_tax < 0 → set NEW.l_tax := 0;
- If NEW.l_tax > 1 → set NEW.l_tax := 1;
- Otherwise, leave the value unchanged.

Example output behavior:

- INSERT INTO lineitem(l_orderkey, l_linenum, l_tax) VALUES (1, 2, -1.2);
- -- After trigger: l_taxl = 0
- UPDATE lineitem SET l_tax = 100 WHERE l_orderkey = 3 and l_item = 6;
- -- After trigger: l_tax = 1

Execution Instructions:

1. PL/pgSQL Script:

- Implement the trigger in a script named: A2SolutionT3.sql.

2. Execution:

- Test the trigger with at least **two scenarios**:
 - Insert a new lineitem with a negative l_tax.
 - Update an existing lineitem with a positive l_tax greater than 1.

3. Execution Output:

- Capture all terminal output including:
 - Successful insert/update operations that do not fire the trigger.
 - Insert/update operations that fire the trigger.
- Save the output in: A2SolutionT3.txt
- Convert the captured output into a **PDF file**: A2SolutionT3.pdf

Submission Instructions:

You are required to submit two files:

1. **SQL script file** (A2SolutionT3.sql) containing:
 - The CREATE TRIGGER statement

- The test INSERT and UPDATE statements to demonstrate trigger behaviour.

2. **Execution report** (A2SolutionT3.pdf) containing:

- Successful creation of the trigger
- Results of all tests
- No syntax or runtime errors

Report Guidelines:

- Your report must include:
 - **Screenshots of the SQL output:** Copy and paste the screen output from your terminal into a document, then save it as a PDF file.
 - **SQL statements processed:** The report must list **all SQL statements** that were executed.
 - Ensure there are **no errors** in the output shown in your report.

SQL Script Requirements:

- At the beginning of your script, include the following:
 - **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
 - **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- The trigger should **only modify I_tax** when a value that is not between 0 and 1 is inserted or updated.
- Include **all SQL statements and execution output** in your report.
- Any submission failing to meet these requirements may receive **no marks**.

Sample solution includes but is not limited to the following:

```
-- Create the trigger function
CREATE OR REPLACE FUNCTION validate_normalize_I_tax()
RETURNS TRIGGER AS
$$
BEGIN
    -- Normalize I_tax value to be within [0, 1]
    IF NEW.L_TAX < 0 THEN
        NEW.L_TAX := 0;
```

```

ELSIF NEW.L_TAX > 1 THEN
    NEW.L_TAX := 1;
END IF;

-- Return the modified row
RETURN NEW;
END;
$$
LANGUAGE plpgsql;

-- Create the trigger on LINEITEM table
CREATE TRIGGER trg_validate_l_tax
    BEFORE INSERT OR UPDATE ON LINEITEM
    FOR EACH ROW
    EXECUTE FUNCTION validate_normalize_l_tax();

tpchr=> \i
D:/Teaching/UOW/CSCI235/Software/PostgreSQL/Scripts/PLpgSQLTriggers/RowTriggerValidateLTax.sql

```

CREATE FUNCTION

CREATE TRIGGER

```
tpchr=> \d lineitem;
```

Table "public.lineitem"

Column	Type	Collation	Nullable	Default
l_orderkey	integer		not null	
l_partkey	integer		not null	
l_suppkey	integer		not null	
l_linenum	integer		not null	
l_quantity	numeric(12,2)		not null	
l_extendedprice	numeric(12,2)		not null	
l_discount	numeric(12,2)		not null	
l_tax	numeric(12,2)		not null	
l_returnflag	character(1)		not null	
l_linestatus	character(1)		not null	

```
l_shipdate    | date          |          | not null |
l_commitdate  | date          |          | not null |
l_receiptdate | date          |          | not null |
l_shipinstruct | character(25) |          | not null |
l_shipmode    | character(10) |          | not null |
l_comment     | character varying(44) |          | not null |
```

Indexes:

```
"lineitem_pkey" PRIMARY KEY, btree (l_orderkey, l_linenumber)
"it2q2idx1" btree (l_shipdate, l_discount)
"it2q2idx2" btree (l_shipmode, EXTRACT(year FROM l_shipdate))
"it2q2idx4" btree (l_linestatus)
"it2q2idx5" btree (l_orderkey, l_linenumber, l_discount, l_shipdate, l_linestatus, l_shipmode)
```

Check constraints:

```
"lineitem_check1" CHECK (l_quantity >= 0::numeric)
"lineitem_check2" CHECK (l_extendedprice >= 0::numeric)
"lineitem_check3" CHECK (l_tax >= 0::numeric)
"lineitem_check4" CHECK (l_discount >= 0.00 AND l_discount <= 1.00)
```

Foreign-key constraints:

```
"lineitem_fkey1" FOREIGN KEY (l_orderkey) REFERENCES orders(o_orderkey)
"lineitem_fkey2" FOREIGN KEY (l_partkey) REFERENCES part(p_partkey)
"lineitem_fkey3" FOREIGN KEY (l_partkey, l_suppkey) REFERENCES partsupp(ps_partkey,
ps_suppkey)
"lineitem_fkey4" FOREIGN KEY (l_suppkey) REFERENCES supplier(s_suppkey)
```

Triggers:

```
trg_validate_l_tax BEFORE INSERT OR UPDATE ON lineitem FOR EACH ROW EXECUTE
FUNCTION validate_normalize_l_tax()
```

tpchr=>

tpchr=> -- Test insert with value < 0 (should be set to 0)

tpchr=> INSERT INTO LINEITEM VALUES

tpchr-> (1, 2, 3, 8, 10.00, 100.00, 0.10, -0.50, 'N', 'O',

tpchr(> '2024-01-01', '2024-01-01', '2024-01-01', 'NONE', 'SHIP', 'Test');

INSERT 0 1

tpchr=>

tpchr=> -- Query to verify the normalized values

tpchr=> SELECT L_ORDERKEY, L_LINENUMBER, L_TAX

tpchr-> FROM LINEITEM

tpchr-> WHERE L_ORDERKEY = 1;

l_orderkey | l_linenum | l_tax

-----+-----+-----

1 | 1 | 1.00

1 | 2 | 0.06

1 | 3 | 0.02

1 | 4 | 0.06

1 | 5 | 0.04

1 | 6 | 0.02

1 | 8 | 0.00

(7 rows)

tpchr=>

tpchr=> -- Test update with value > 1 (should be set to 1)

tpchr=> UPDATE LINEITEM

tpchr-> SET L_TAX = 1.5

tpchr-> WHERE L_ORDERKEY = 1 AND L_LINENUMBER = 8;

UPDATE 1

tpchr=>

tpchr=> -- Query to verify the normalized values

tpchr=> SELECT L_ORDERKEY, L_LINENUMBER, L_TAX

tpchr-> FROM LINEITEM

tpchr-> WHERE L_ORDERKEY = 1;

l_orderkey | l_linenum | l_tax

-----+-----+-----

1 | 1 | 1.00

1 | 2 | 0.06

1 | 3 | 0.02

1 | 4 | 0.06

1 | 5 | 0.04

1 | 6 | 0.02

1 | 8 | 1.00

(7 rows)

tpchr=>

tpchr=> -- Delete the test data

tpchr=> delete from lineitem where l_orderkey = 1 and l_linenum = 8;

DELETE 1

tpchr=>

Submissions

This assignment is due by 9:00 pm (2100 hours) Sunday, 10 May 2026, Singapore time.

Submit the files **solution1.sql**, **solution1.pdf**, **solution2.sql**, **solution2.pdf**, **solution3.sql**, and **solution3.pdf** through Moodle in the following way:

- 1) Zip all the files (solution1.sql, solution1.pdf, solution2.sql, solution2.pdf, solution3.sql, and solution3.pdf) into one zipped folder. Name your zipped file as YourPUStudentNumber-A3)
- 2) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 3) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 4) When successfully logged in, select a site CSCI235 (SP226) Database Systems
- 5) Scroll down to a section Submissions of Assignments
- 6) Click at Submit your Assignment 1 here link.
- 7) Click at a button Add Submission
- 8) Move the zipped file created in Step 1 above into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 9) Click at a button Save changes,
- 10) Click at check box to confirm authorship of a submission,
- 11) When you are satisfied, remember to click at a button Submit assignment.

A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.

Assignment 2 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification